Software Requirements Specification (SRS)

Project Name: BonPlan.ai
Document Status: Approved for User Stories Generation
Product Vision: Tell us When. We Tell the How.

1.0 Functional Requirements (FRs)

1.1 Authentication & User Management

- **FR-1.1.1:** The system shall allow users to create an account and authenticate using standard email/password and OAuth (e.g., Google).
- **FR-1.1.2:** The system shall allow users to log into their account using standard email/password and OAuth (e.g., Google).
- **FR-1.1.3:** The system shall support Role-Based Access Control (RBAC) for shared trips, assigning users "Owner," "Editor," or "Viewer" permissions.

1.2 Data Ingestion, Initialization & State Management

- **FR-1.2.1:** The system shall accept manual "Smart Anchors" requiring a minimum of three fields: `Title`, `Start_Time` (ISO 8601), and `End_Time` (ISO 8601), and flag them with an immutable `is_anchor = true` boolean.
- **FR-1.2.2:** The system shall accept baseline initialization parameters including Source, Destination, Start Date, End Date, and an explicit "Pacing Preference" (e.g., Relaxed, Action-Packed).
- **FR-1.2.3:** The system shall ingest and process free-form conversational context (e.g., "Make this a chill food tour") and append it to the LLM system prompt for baseline generation.
- **FR-1.2.4:** The system shall accept and strictly enforce user-defined personalization constraints, including daily budget limits, dietary restrictions, and accessibility requirements.
- **FR-1.2.5:** The system shall support "Recurring Anchors" (e.g., a 7:00 AM daily gym session), propagating the anchor across all active days in the trip timeline.
- **FR-1.2.6:** The backend logic shall calculate available "whitespace" (free time slots) between anchors by subtracting an adjustable transit buffer (default: 30 minutes) from the start and end times of adjacent anchors.

1.3 Agentic Processing (The ReAct Loop)

- **FR-1.3.1:** The orchestration layer shall inject the calculated whitespace parameters and user personalization constraints into the LLM system prompt prior to generation.
- **FR-1.3.2:** The agent shall utilize a `Search_Places` tool to retrieve potential activities within a dynamically calculated radius of the previous geographic coordinate.
- **FR-1.3.3:** The agent shall verify the `open_now` or specific operating hours of a proposed location via an external API before appending it to the itinerary schema.
- **FR-1.3.4:** The agent shall utilize a `Calculate_Distance` tool to verify that transit time between Event A and Event B does not violate the start time of any downstream Smart Anchor.
- **FR-1.3.5:** The agent shall utilize a `Check_Weather` tool to verify outdoor feasibility and swap to indoor alternatives if adverse weather is detected during the active generation block.
- **FR-1.3.6:** The agent shall retrieve and embed actionable booking URLs (deep-links) for flights, hotels, and ticketed landmarks when available.

1.4 User Interface & Interaction (The Canvas)

- **FR-1.4.1:** The React frontend shall render the itinerary as a vertically scrollable, time-blocked calendar canvas, with a toggleable visual Map View.
- **FR-1.4.2:** The frontend shall allow users to drag an `is_anchor = false` event to modify its start or end time, triggering a backend "Ripple Effect" recalculation that pushes subsequent unanchored events forward.
- **FR-1.4.3:** If the Ripple Effect calculation collides with a Smart Anchor, the system shall halt the cascade, flag the conflicting event in the UI, and prompt the user to compress or delete an activity.
- **FR-1.4.4:** The system shall provide a "Specific Regeneration" action, allowing users to highlight a single event and request a swap. The backend must replace it with an geographically and temporally viable alternative without altering the rest of the timeline.
- **FR-1.4.5:** The system shall allow users to manually promote any AI-generated event to a Smart Anchor (`is_anchor = true`) via a UI toggle.

1.5 Multiplayer & Collaboration (New)

- **FR-1.5.1:** The system shall generate a secure, unique URL to invite collaborators to a specific Trip ID.
- **FR-1.5.2:** The UI shall feature a voting module (upvote/downvote) on generated activities, automatically resorting or swapping activities based on the group's consensus score.
- **FR-1.5.3:** The system shall allow the creation of "Split Events," permitting concurrent, overlapping activities for different group members on the same timeline.

1.6 Live Concierge & Post-Generation (New)

- **FR-1.6.1:** The system shall support exporting the finalized itinerary into a static, printer-friendly PDF format.
- **FR-1.6.2:** The system shall cache the active itinerary in the frontend client (e.g., via Service Workers/LocalStorage) to allow offline viewing without an active network connection.
- **FR-1.6.3:** The backend shall support an asynchronous job to send a "Morning Summary" push notification/email containing the day's schedule and localized weather.
- **FR-1.6.4:** The system shall listen for external flight delay webhooks and proactively send a push notification offering a one-click automated itinerary compression to absorb the lost time.

2.0 Non-Functional Requirements (NFRs)

2.1 Performance & Latency

- **NFR-2.1.1 (Full Generation):** The system shall complete an end-to-end trip generation (spanning up to 7 days) and render the initial JSON payload to the frontend in under 15 seconds.
- **NFR-2.1.2 (Surgical Swap):** The system shall process a "Specific Regeneration" (swapping a single 2-hour activity) and render the UI update in under 10 seconds.
- **NFR-2.1.3 (Streaming):** During any generation exceeding 2 seconds, the backend shall stream Server-Sent Events (SSE) to the frontend indicating the agent's current thought process (e.g., "Verifying museum hours...").

2.2 Cost Optimization & Safety Guardrails

- **NFR-2.2.1 (Loop Prevention):** The LangChain/LlamaIndex orchestrator shall enforce a strict `max_iterations = 5` limit per reasoning loop to prevent infinite API polling.
- **NFR-2.2.2 (Fallback State):** If the agent fails to find a valid location fitting the constraints within the maximum iterations, it shall exit the loop and return a standardized `Constraint_Negotiation` JSON payload, asking the user to expand their search radius or time block.

2.3 Scalability & Concurrency

- **NFR-2.3.1:** The FastAPI backend shall support fully asynchronous API calls, handling up to 50 concurrent trip generation requests without dropping connections.
- **NFR-2.3.2:** In Multiplayer Collaborative Mode, the database must resolve conflicting user votes or simultaneous drag-and-drop actions using a Last-Write-Wins (LWW) concurrency model to prevent timeline corruption.

3.0 Security, Privacy & Compliance

- **NFR-3.1.1:** User-provided Smart Anchors (which may contain PII like exact hotel addresses or private meeting locations) shall be sanitized and must not be logged in persistent LLM training data pipelines.
- **NFR-3.1.2:** The FastAPI backend shall implement API rate limiting (maximum 10 full generations per IP address per hour) to mitigate malicious token-draining attacks.
- **NFR-3.1.3:** JWT (JSON Web Tokens) or secure session cookies shall be strictly utilized to validate API requests for any Trip ID that is not flagged as "Public."

4.0 System Constraints & Assumptions

- **SC-1:** The system assumes a maximum LLM context window of 128k tokens; historical conversational context in the "Trip For All" mode will be truncated using a rolling window if it exceeds this limit.
- **SC-2:** Real-time routing accuracy is strictly constrained by the rate limits and geographical coverage of the chosen Maps/Places API provider.